

超标量

Superscalar Processors

mano@cs.auckland.ac.nz

Superscalar Processors

- The pipelined processor we studied so far is able to issue one instruction per clock cycle
- A superscalar processor has multiple concurrent functional units, and therefore is able to issue multiple instructions per clock cycle

Superscalar Processors

- Requires instructions to be grouped for concurrent issuing. Grouping decisions take care of the pipeline hazards.
 - Static multiple issue: Compiler forms issue packets. The issue packet can be thought of as a long instruction (VLIW: very long instruction word; EPIC: explicitly parallel instruction computer)
 - Dynamic multiple issue: Hardware makes grouping decisions.

Software Pipelining

- We look at an example of software pipelining whereby the compiler groups (or schedules) instructions on multiple functional units.
- We use the MIPS R10K processor as an example.

MIPS R10K Processor

- MIPS R10K is a superscalar processor with 4 functional units.
- The processor is capable of doing two floating-point operations and two integer operations per clock cycle.
 - One of these operations could be a memory access (a load or a store).
 - One of the integer operations could be a branch.

Instruction Attributes

Instruction	Latency	Repeat rate
Integer add/sub/logical/branch	1	1
Integer load/store	2	1
Floating-point load/store	3	1
Floating-point add/sub/multiply	2	1
Floating-point multiply-add	3	1
Floating-point division	19	21

实际所需时间。

为了基准
评分专门优化

Latency

- The latency of an instruction refers to the number of cycles an instruction takes to execute.
- Since the processor is pipelined, the next instruction could start executing in the following cycle while the previous one is being executed.

Repeat rate

- The repeat rate of an instruction refers to the number of cycles before which another instruction of the same type cannot begin execution.
- According to the instruction attributes table, all instructions except floating-point division could be issued every cycle. A floating-point division, however, can be issued only every 21st cycle.

DAXPY

```
for (i = 0; i < N; i++) {  
    y[i] = a * x[i] + y[i];  
}
```

This multiplies a scalar a by a vector x and adds vector y to it (ax *PLUS* $y = axpy$). D for double precision.

Assume that a is in $\$s1$, start address of x in $\$s2$, and start address of y in $\$s3$.

DAXPY

```
addiu $s4, $s2, 1000      ; set up x end pointer
```

```
addiu $s6, $s3, 1000      ; set up y end pointer
```

```
Loop:  lf $f2, 0($s2)       ; load x value in f2
```

```
      lf $f3, 0($s3)       ; load y value in f3
```

```
      madd $f3, $s1, $f2    ;
```

```
      sf $f3, 0($s3)       ; store f3 to y
```

```
      addiu $s2, $s2, 4     ; increment x pointer
```

```
      addiu $s3, $s3, 4     ; increment y pointer
```

```
      bne $s2, $s4, Loop    ; back to the beginning of loop
```

Instruction Scheduling

Cycle	FU #1 (FP)	FU #2 (INT)	FU #3 (INT)	FU #4 (FP)
1	lf x_1	addiu (x_1)		
2	lf y_1			
3				
4				
5				madd x_1, y_1
6				
7				
8	sf y_1	addiu (y_1)	bne	
9				

addiu $\$s4, \$s2, 1000$

addiu $\$s6, \$s3, 1000$

Loop: lf $\$f2, 0(\$s2)$

lf $\$f3, 0(\$s3)$

madd $\$f3, \$s1, \$f2$

sf $\$f3, 0(\$s3)$

addiu $\$s2, \$s2, 4$

addiu $\$s3, \$s3, 4$

bne $\$s2, \$s4, \text{Loop}$

late

What is the FU utilization here?

Exercises

- Unroll the DAXPY loop once and schedule the instructions
- Unroll the DAXPY loop twice and schedule the instructions

DAXPY unrolled once

```
for (i = 0; i < N/2; i+=2) {  
    y[i] = a * x[i] + y[i];  
    y[i+1] = a * x[i+1] + y[i+1];  
}
```

Cycle	FU #1 (FP)	FU #2 (INT)	FU #3 (INT)	FU #4 (FP)
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				
14				

addiu \$s4, \$s2, 1000

addiu \$s6, \$s3, 1000

Loop:

If \$f2, 0(\$s2)

If \$f3, 0(\$s3)

madd \$f3, \$s1, \$f2

sf \$f3, 0(\$s3)

addiu \$s2, \$s2, 4

addiu \$s3, \$s3, 4

If \$f4, 0(\$s2)

If \$f5, 0(\$s3)

madd \$f5, \$s1, \$f4

sf \$f5, 0(\$s3)

addiu \$s2, \$s2, 4

addiu \$s3, \$s3, 4

bne \$s2, \$s4, Loop

What is the FU utilization here?

Cycle	FU #1 (FP)	FU #2 (INT)	FU #3 (INT)	FU #4 (FP)
1	lf x_1	addiu (x_1)		
2	lf y_1			
3	lf x_2	addiu (x_2)		
4	lf y_2			
5				madd x_1, y_1
6				
7				madd x_2, y_2
8	sf y_1	addiu (y_1)		
9				
10	sf y_2	addiu (y_2)	bne	
11				
12				
13				
14				

Loop:

```

lf $f2, 0($s2)
lf $f3, 0($s3)
madd $f3, $s1, $f2
sf $f3, 0($s3)
addiu $s2, $s2, 4
addiu $s3, $s3, 4

lf $f4, 0($s2)
lf $f5, 0($s3)
madd $f5, $s1, $f4
sf $f5, 0($s3)
addiu $s2, $s2, 4
addiu $s3, $s3, 4
bne $s2, $s4, Loop

```

What is the FU utilization here?

Cycle	FU #1 (FP)	FU #2 (INT)	FU #3 (INT)	FU #4 (FP)
1	lf x_1	addiu (x_1)		
2	lf y_1			
3	lf x_2	addiu (x_2)		
4	lf y_2			
5				madd x_1, y_1
6				
7				madd x_2, y_2
8	sf y_1	addiu (y_1)		
9				
10	sf y_2	addiu (y_2)	bne	
11				
12				
13				
14				

Loop:

```

addiu $s4, $s2, 1000
addiu $s6, $s3, 1000

lf $f2, 0($s2)
lf $f3, 0($s3)
madd $f3, $s1, $f2
sf $f3, 0($s3)
addiu $s2, $s2, 4
addiu $s3, $s3, 4

lf $f4, 4($s2)
lf $f5, 4($s3)
madd $f5, $s1, $f4
sf $f5, 0($s3)
addiu $s2, $s2, 8
addiu $s3, $s3, 8
bne $s2, $s4, Loop

```

Cycle	FU #1 (FP)	FU #2 (INT)	FU #3 (INT)	FU #4 (FP)
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				
14				

Cycle	FU #1 (FP)	FU #2 (INT)	FU #3 (INT)	FU #4 (FP)
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				
14				

Further Work

- Read Chapter 4 of the book: Computer Organization and Design – The Hardware Software Interface RISC-V edition, 2nd edition (2020)
- Read https://en.wikipedia.org/wiki/Software_pipelining

¿?

Caches 缓存

- A cache is a small local memory holding items of interest
 - Items of interest include items we used in the recent past and items we may use in the future.
 - Example: a browser cache that holds the web content you looked at recently
- Why is such a cache useful?
 - Avoiding re-fetching items over a (slow) network
 - Processors are a lot faster than memory